



Retail Infrastructure Platform

Complete Development Guide v2.0

100M Customers	99.99% Uptime	15M+ Stores	64 Kafka Topics
----------------	---------------	-------------	-----------------

Backend · Customer App · Retailer · Admin · Infrastructure
March 2026 | Audited & Corrected

Table of Contents

PART 1	Platform Overview & Architecture	pg 3
PART 2	Backend — Monorepo & Shared Packages	pg 6
PART 3	Backend — All 18 Microservices (Complete)	pg 10
PART 4	Database Layer (PostgreSQL + Redis + Kafka + ES + TimescaleDB + ClickHouse)	pg 18
PART 5	Customer Mobile App (React Native)	pg 28
PART 6	Retailer Dashboard + Staff App + Electron Kiosk	pg 34
PART 7	Admin & Chain HQ Platform	pg 40
PART 8	Infrastructure & DevOps (Terraform + K8s + CI/CD + Observability)	pg 44
PART 9	18-Week Execution Roadmap	pg 52
PART 10	Pre-Pilot Go-Live Checklist	pg 58



Platform Overview & Architecture

1.1 What Zippyra Is

Zippyra is a unified retail infrastructure platform bringing self-checkout, AI-powered inventory management, and multi-layer anti-theft to offline stores across India. Three surfaces — Customer App, Retailer Dashboard, and Admin/Chain HQ — share ONE backend, separated by JWT user_type claims.

Scale Targets

Metric	Target / Decision
Customers	100 Million active users
Stores	15 Million+ offline stores across India
Payment Uptime	99.99% — < 52 minutes downtime per year
Cart Scan P99	< 100ms — Redis-only fast path, zero PostgreSQL in happy path
Barcode Lookup P99	< 50ms — Redis SKU cache hit, fallback to read replica
Kafka	64 topics total (28 customer + 8 retailer + 6 admin + 22 new/shared), 256 partitions on cart.item_scanned
PostgreSQL tables	46 tables across 12 service groups
Redis clusters	6 clusters (sessions, cart, catalog, payments, realtime, analytics)
TimescaleDB hypertables	8 hypertables + 20 continuous aggregates
ClickHouse tables	9 analytics tables
Elasticsearch indexes	3 indexes (products, stores, orders_search, support_tickets, users_search)
WatermelonDB tables	11 tables (v11 schema, migrations v1→v4)
Team (Phase 0)	1–2 engineers for pilot → scale with revenue

1.2 Three-Layer Architecture

LAYER 1 Customer App	React Native (Expo SDK 51) + WatermelonDB offline-first. Core flow: Login → Scan Entrance QR → Scan Products → Pay → Show Exit QR → Gate opens.
LAYER 2 Retailer Side	Next.js 14 PWA (retailer.zippyra.com) + React Native Staff App + Electron Kiosk. Inventory state machine, 7-layer anti-theft, GRN workflow, ERP integrations, ClickHouse analytics.
LAYER 3 Admin / HQ	Next.js 14 at admin.zippyra.com + hq.zippyra.com. 10-step store onboarding, 8 hardware device types via MQTT/AWS IoT Core, DPDP/PCI-DSS/GSTN/RBI compliance.

1.3 Full Technology Stack

Component	Technology	Notes
Backend Language	Go 1.22	All 18 microservices, single monorepo
Customer App	React Native + Expo SDK 51	TypeScript, Expo Router, EAS Build
Retailer Dashboard	Next.js 14 + Tailwind CSS	TypeScript, App Router, PWA

Admin Platform	Next.js 14 (monorepo: admin + hq)	TypeScript, shadcn/ui
Primary DB	PostgreSQL 16 — AWS RDS Multi-AZ	46 tables, golang-migrate, pgxpool
Cache Layer	Redis 7 — AWS ElastiCache	6 clusters, cluster-mode at Phase 1
Message Queue	Apache Kafka — AWS MSK	64 topics, 12 brokers prod, store_id partition key
Search	AWS OpenSearch (ES 8 compat)	5 indexes: products, stores, orders, tickets, users
Analytics DB	ClickHouse Cloud	9 tables, real-time retail analytics
Time-Series DB	TimescaleDB (Timescale Cloud)	8 hypertables + 20 continuous aggregates
Offline Sync	WatermelonDB (SQLite)	11 tables, schema v11, migrations v1→v4
Payments	Razorpay (primary) + Cashfree (fallback)	HMAC webhook, circuit breaker
OTP/SMS	Twilio Verify	Async delivery, 5-layer rate limit
Push	Firebase FCM + WhatsApp Business API	Receipt + exit QR delivery
IoT/Hardware	AWS IoT Core + MQTT	8 device types, heartbeat monitoring
Containers	AWS EKS (Kubernetes)	Namespace per env, HPA, zero-downtime
IaC	Terraform	8 modules, pilot + production environments
CI/CD	GitHub Actions	4 workflows: lint-test, staging, prod, eas-build
Observability	OTel + Prometheus + Jaeger + Grafana	3 dashboards, PagerDuty alerts
Auth	Ed25519 JWT	24h access + 30-day refresh, Redis blacklist
Object Storage	AWS S3	Invoices, product images, store media



Backend — Monorepo & Shared Packages

2.1 Monorepo Folder Structure (Complete)

■ All 18 Go microservices in ONE monorepo. Single go.mod. Deploy independently as Docker containers. Separated by JWT user_type — not by network or codebase.

zippyra/platform/

■ ■ ■ ■ go.mod

■ ■ ■ ■ go.sum

■ ■ ■ ■ Makefile

■ ■ ■ ■ .env.example

■ ■ ■ ■ docker-compose.yml

■ ■ ■ ■ shared/

■ ■ ■ ■ db/

■ ■ ■ ■ db.go

■ ■ ■ ■ redis/

■ ■ ■ ■ redis.go

■ ■ ■ ■ scripts.go

■ ■ ■ ■ kafka/

■ ■ ■ ■ kafka.go

■ ■ ■ ■ topics.go

■ ■ ■ ■ jwt/

■ ■ ■ ■ jwt.go

■ ■ ■ ■ types.go

■ ■ ■ ■ middleware/

■ ■ ■ ■ middleware.go

■ ■ ■ ■ auth.go

■ ■ ■ ■ ratelimit.go

■ ■ ■ ■ otel/

■ ■ ■ ■ otel.go

■ ■ ■ ■ errors/

■ ■ ■ ■ errors.go

■ ■ ■ ■ validator/

■ ■ ■ ■ validator.go

■ ■ ■ ■ services/

■ ■ ■ ■ auth/

■ ■ ■ ■ main.go

■ ■ ■ ■ handler_otp.go

■ ■ ■ ■ handler_token.go

■ ■ ■ ■ handler_version.go

■ ■ ■ ■ otp_hash.go

■ ■ ■ ■ twilio.go

Go monorepo root — github.com/zippyra/platform

Single module: require pgx, redis, kafka-go, golang-jwt, chi, zerolog, otel, twilio

Targets: build, test, lint, migrate-up, migrate-down, seed, docker-compose-up

All required env vars documented

Local dev: postgres, redis×6, kafka, zookeeper, opensearch, jaeger

Packages imported by ALL 18 services

pgxpool factory: RDS Proxy compat, 20 max conns, 3-retry, 5s timeout, slow query tracer

Clients{} with 6 cluster fields, ClusterFor() routing, Lua scripts, GatewayCircuitBreaker

OTPRateLimitScript, StoreCapacityIncrementScript, InventoryHoldScript, ExitQRUseScript

Producer (store_id partition key enforced), Consumer (DLQ after 3 retries), Message envelope

ALL 64 topic constants — never hardcode topic names in services

Ed25519 sign/verify,

Claims{UserID, UserType, StoreID, DeviceID, ChainID, Role}, Revoke()

UserType constants: CUSTOMER, STAFF, CHAIN_HQ, ZIPPYRA_ADMIN, SUPPORT_AGENT

Standard() chain: RequestID→logging→panic recover→CORS→timeout

JWTAuth(), CustomerOnly(), StoreScopeRequired(), ClaimsFromContext()

Redis token bucket rate limiter middleware

otel.Init(serviceName) — called in every main.go. OTLP exporter to Jaeger.

Typed error codes: ErrNotFound, ErrUnauthorized, ErrRateLimit, ErrGateway, etc.

Phone regex, UUID check, GST rate bounds, barcode format validators

18 microservices — each is a standalone binary

Port :8080

Router setup, all handlers, Twilio client init

handleOTPSend (10 steps), handleOTPVerify (10 steps)

handleRefresh, handleLogout, handleMe

handleVersionCheck — reads app_versions table

generateOTP(), hashOTP(otp, phone) — SHA-256 with phone salt

sendOTPViaTwilio() — Twilio Verify SDK wrapper

■ ■■■ store/	Port :8082
■ ■ ■■■ main.go	Router: /v1/store/bind, /v1/store/exit, /v1/store/session, /v1/store/{id}/info
	QR parse→hash→DB lookup→is_used check→expiry→capacity
■ ■ ■■■ handler_bind.go	(Lua)→JWT upgrade
■ ■ ■■■ handler_exit.go	Decrement capacity, delete session, revoke JWT
■ ■ ■■■ qr.go	parseQRToken(), hashToken() — SHA-256 of raw token
■ ■■■ catalog/	Port :8083
	Router: /v1/catalog/barcode/{barcode}, /v1/catalog/search, /v1/catalog/sync
■ ■ ■■■ main.go	Redis cache hit (<10ms) → PostgreSQL read replica fallback → write-back cache
■ ■ ■■■ handler_barcode.go	Elasticsearch query with category/brand/price filters
■ ■ ■■■ handler_search.go	Delta sync: pull products WHERE sync_seq > lastSeen, ETag conditional
■ ■ ■■■ handler_sync.go	Consume catalog.product_updated Kafka → index to OpenSearch
■ ■ ■■■ es_indexer.go	Port :8084
■ ■■■ cart/	Router + inventory reconciler goroutine (10s ticker)
■ ■ ■■■ main.go	Full Redis fast path: SKU lookup → hold (Lua) → cart load → item add → offers → save
	GET /v1/cart, DELETE /v1/cart/item/{id}, DELETE /v1/cart
■ ■ ■■■ handler_scan.go	evaluateOffers() — pure Go, no I/O. PERCENTAGE, BOGO, BUNDLE, COMBO rule types.
■ ■ ■■■ handler_cart.go	runInventoryReconciler() — Redis holds → PostgreSQL stock every 10s
■ ■ ■■■ offer_engine.go	Port :8085
■ ■ ■■■ reconciler.go	Router: /v1/payment/initiate (authenticated), /v1/payment/webhook/razorpay (public)
■ ■■■ payment/	Checkout lock SETNX → cart snapshot → price revalidate → idempotency insert → gateway
■ ■ ■■■ main.go	HMAC-SHA256 verify FIRST → ON CONFLICT DO NOTHING → async process → 200 in <200ms
■ ■ ■■■ handler_initiate.go	GET /v1/payment/status/{id}
■ ■ ■■■ handler_webhook.go	createRazorpayOrder() — POST https://api.razorpay.com/v1/orders
■ ■ ■■■ handler_status.go	createCashfreeOrder() — fallback when circuit open
■ ■ ■■■ gateway_razorpay.go	Wraps shared/redis GatewayCircuitBreaker, per-gateway state
■ ■ ■■■ gateway_cashfree.go	Port :8086
■ ■ ■■■ circuit_breaker.go	Health HTTP server + Kafka consumer init
■ ■■■ order/	handlePaymentConfirmed() — load cart → create order → order_items → exit token → S3
■ ■ ■■■ main.go	generateInvoicePDF() — async S3 upload (gofpdf in production)
■ ■ ■■■ consumer.go	Port :8087
■ ■ ■■■ invoice.go	Router: /v1/exit/validate (authenticated), /v1/exit/rfid-confirm (device call)
■ ■■■ exit-validation/	Ed25519 verify → store scope → SET NX → exit_preauth check → RFID check → log → gate open
■ ■ ■■■ main.go	storeHasRFIDPad() — devices table check with Redis cache (5 min TTL)
■ ■ ■■■ handler_validate.go	Port :8088
■ ■ ■■■ rfid.go	Kafka consumers: notification.payment_receipt, notification.otp
■ ■■■ notification/	sendWhatsAppReceipt() — WhatsApp Business API with receipt + exit QR
■ ■ ■■■ main.go	sendFCMPush() — Firebase Admin SDK, payment_confirm + exit_qr_ready notifications
■ ■ ■■■ whatsapp.go	sendSMSOTP() — Twilio SMS fallback when WhatsApp unavailable
■ ■ ■■■ fcm.go	Port :8089
■ ■ ■■■ sms.go	Kafka consumer: loyalty.points_earned + loyalty.tier_changed
■ ■■■ loyalty/	
■ ■ ■■■ main.go	

■ ■ ■■ handler_earn.go

■ ■ ■■ handler_redeem.go

■ ■ ■■ tier.go

■ ■■ inventory/

■ ■ ■■ main.go

■ ■ ■■ handler_grn.go

■ ■ ■■ handler_adjust.go

■ ■ ■■ state_machine.go

■ ■■ warehouse/

■ ■ ■■ main.go

■ ■ ■■ handler_po.go

■ ■ ■■ handler_transfer.go

■ ■■ analytics/

■ ■ ■■ main.go

■ ■ ■■ clickhouse.go

■ ■ ■■ handler_dashboard.go

■ ■■ support/

■ ■ ■■ main.go

■ ■ ■■ handler_ticket.go

■ ■ ■■ handler_message.go

■ ■■ retailer-auth/

■ ■ ■■ main.go

■ ■ ■■ handler_kiosk.go

■ ■■ admin-auth/

■ ■ ■■ main.go

■ ■ ■■ totp.go

■ ■■ chain-hq/

■ ■ ■■ main.go

■ ■ ■■ erp.go

■ ■■ admin-store/

■ ■ ■■ main.go

■ ■ ■■ onboarding.go

■ ■ ■■ hardware.go

■ ■■ compliance/

■ ■ ■■ main.go

■ ■ ■■ dpdp.go

■ ■ ■■ gstn.go

■■■ cmd/

■ ■■ admin-cli/

Calculate points (1 point per ■10 base, multiplier by tier) → insert loyalty_ledger

POST /v1/loyalty/redeem — atomic deduct from loyalty_cards.points_balance

evaluateTierUpgrade() — publish loyalty.tier_changed if tier changes

Port :8090 | JWT: STAFF

Router: /v1/inventory/grn, /v1/inventory/low-stock, /v1/inventory/adjust

Receive GRN → update stock table → publish inventory.low_stock if needed

Manual stock adjustment → append inventory_movements record

AVAILABLE → RESERVED → SOLD → RETURNED state transitions

Port :8091 | JWT: STAFF

Router: /v1/warehouse/po, /v1/warehouse/transfer

Purchase order CRUD, vendor lookup, publish warehouse.grn_discrepancy

Store-to-store transfer → publish warehouse.transfer_dispatched

Port :8092 | JWT: CHAIN_HQ

Kafka consumers: order.completed, cart.item_scanned, exit.validated → ClickHouse write

ClickHouse HTTP client, batch writes, sales_summary + peak_hour_heatmap

GET /v1/analytics/store/{id} — real-time KPIs from ClickHouse

Port :8093 | JWT: ALL

Router: /v1/support/ticket (CRUD), /v1/support/ticket/{id}/message

Create ticket → publish support.ticket_created Kafka event

Append message → notify agent via FCM

Port :8094 | JWT: STAFF

POST /v1/staff/otp/send, /v1/staff/otp/verify — issues STAFF JWT with role claim

POST /v1/staff/kiosk/session — Electron kiosk device session

Port :8095 | JWT: ZIPPYRA_ADMIN

POST /v1/admin/login (SSO), /v1/admin/2fa/verify (TOTP), /v1/admin/logout

TOTP using github.com/pquerna/otp — 30s window, backup codes

Port :8096 | JWT: CHAIN_HQ

Multi-store config API, ERP webhook endpoints, chain-wide reports

ERP sync handlers: SAP, Tally, Busy, Unicommerce — publish erp.sync_jobs Kafka

Port :8097 | JWT: ZIPPYRA_ADMIN

10-step onboarding API, device registration (MQTT/IoT Core), compliance checks

Wizard steps 1–10 with state machine:

DRAFT→CONFIGURED→ACTIVE

Device registration: RFID_PAD, QR_SCANNER, KIOSK, GATE_SENSOR, WEIGHT_SCALE, RECEIPT_PRINTER, CAMERA, BLE_BEACON

Port :8098 | JWT: SYSTEM

Kafka consumer: account.deletion_requested → purge PII across all services

30-day deletion schedule, PII field nullification, audit trail

GSTN reconciliation endpoint, HSN rate sync

CLI tools — standalone binaries

- ■ ■■ main.go
- migrations/
- ■■ 000001_auth_service.up.sql
- ■■ 000002_store_service.up.sql
- ■■ 000003_catalog_service.up.sql
- ■■ 000004_cart_payment_order_exit.up.sql
- ■■ 000005_loyalty_notification_devices_warehouse.up.sql
- ■■ 000006_support_admin_staff.up.sql
- ■■ 000007_payment_audit.up.sql
- ■■ 000008_loyalty_ledger.up.sql
- ■■ 000009_refunds.up.sql
- ■■ 000010_warehouse_stock.up.sql
- ■■ 000011_user_notification_prefs.up.sql
- ■■ 000012_coupons.up.sql
- ■■ 000013_offers.up.sql
- ■■ 000014_seed_hsn_gst_rates.up.sql
- infra/
 - ■■ modules/vpc/
 - ■■ modules/rds/
 - ■■ modules/elasticache/
 - ■■ modules/msk/
 - ■■ modules/eks/
 - ■■ modules/iam/
 - ■■ modules/secrets/
 - ■■ modules/s3/
 - ■■ modules/opensearch/
 - ■■ modules/iot/
 - ■■ environments/pilot/
 - ■■ environments/production/
- .github/workflows/
- ■■ lint-and-test.yml
- ■■ deploy-staging.yml
- ■■ deploy-production.yml
- ■■ eas-build.yml
- k8s/
 - ■■ base/
 - ■■ overlays/pilot/
 - ■■ overlays/production/
- scripts/
- ■■ seed_pilot_store.sh

Commands: onboard, catalog-import, seed-hsn, seed-offer-rules, verify-deps

golang-migrate SQL files — run in order

users, auth_events, login_attempts, email_verifications, app_versions

stores, store_hours, store_qr_tokens, beacon_verifications, vendors, admin_stores, chains

hsn_gst_rates, categories, products (+sync_seq trigger), pricing_rules, offer_rules, combo_rules, recipes, planogram_map, aisle_layout, shelf_positions

cart_sessions, cart_items, stock, damage_log, orders, order_items (immutable rules), payments, payment_webhook_events, refund_requests, tokenized_cards, reconciliation_log, exit_validation_log

loyalty_tiers, loyalty_cards, loyalty_transactions, notification_templates, notification_log, fcm_tokens, devices, device_events, purchase_orders, grn_records, inventory_movements, store_transfers

support_tickets, support_ticket_messages, admin_actions (append-only), staff_members, customer_profiles, data_deletion_requests + FK backfill

payment_audit_log (immutable, forever retention), approval_workflow

loyalty_ledger, loyalty_tier_history

refunds table, refund_requests (FK to orders), override_log (exit)

warehouse_stock, transfers (cross-service FK)

user_notification_prefs, notification_delivery_log

coupons table (GROUP 4 — Cart Service)

offers table (separate from offer_rules — GROUP 3 Catalog)

Seed ~80 common retail HSN codes. Full 12,000+ from GSTN portal CSV.

Terraform IaC

VPC, public/private/db subnets, NAT gateway, IGW, security groups

PostgreSQL RDS Multi-AZ, parameter group, subnet group, backup 7d

6 Redis clusters with per-cluster eviction policy and node sizing

Kafka MSK, 12 brokers prod, topic configs, SASL auth

EKS cluster, managed node groups, IRSA, cluster autoscaler

Service accounts, policies, KMS keys for secrets

AWS Secrets Manager — DB, Redis, Kafka, JWT keys, API keys

3 buckets: invoices (private), products (private), media (CloudFront CDN)

AWS OpenSearch domain, 5 indexes, ILM policies

AWS IoT Core: thing registry, MQTT topics, device policies, shadow

t4g instances, 3-node MSK, single-AZ Redis, 1 EKS node group

r6g instances, 12-node MSK, cluster-mode Redis, multi-AZ EKS

CI/CD

Every PR: go vet, golangci-lint, go test (Docker Compose), coverage ≥70% auth/payment/exit

Merge to main: ECR push, EKS rollout staging, smoke tests

Manual + 1 approval: ECR push, EKS rollout prod, auto-rollback if error >1% in 5min

Tagged v* releases: EAS Build Android + iOS, Fastlane submit

Kubernetes manifests (generated by Helm or Kustomize)

Deployment, Service, ConfigMap, HPA templates per service

Pilot-specific resource limits and replica counts

Production autoscaling config

Operational scripts

One-shot: run zippyra-admin onboard + catalog-import + seed-hsn + seed-offer-rules

■ ■■■ load_test.js

k6 script: 50 concurrent scan→pay→exit flows per store

■ ■■■ rotate_keys.sh

Rotate ED25519 keypair + update Secrets Manager + rolling pod restart

■ ■■■ import_hsn_full.sh

Download GSTN HSN master CSV + run full 12,000-code import



Backend — All 18 Microservices (Complete)

3.1 Service Registry

Service	Port	JWT Type(s)	HTTP Endpoints	Kafka Consumed	Kafka Produced
auth-service	:8080	PUBLIC / CUSTOMER	POST /otp/send, POST /otp/verify, POST /refresh, POST /logout, GET /me, GET /version-check	—	auth.otp_sent, auth.login_success, auth.login_failed, auth.account_locked, auth.session_created
store-service	:8082	CUSTOMER	POST /store/bind, POST /store/exit, GET /store/session, GET /store/{id}/info, GET /store/{id}/hours	—	store.session_started, store.capacity_alert
catalog-service	:8083	CUSTOMER / STAFF	GET /catalog/barcode/{barcode}, GET /catalog/search, POST /catalog/sync	catalog.product_updated	catalog.product_updated (ES indexer)
cart-service	:8084	CUSTOMER	POST /cart/scan, GET /cart, DELETE /cart/item/{id}, DELETE /cart	—	cart.item_scanned, cart.item_removed, cart.abandoned, cart.checkout_initiated
payment-service	:8085	CUSTOMER + PUBLIC webhook	POST /payment/initiate, GET /payment/status/{id}, POST /payment/webhook/razorpay	—	payment.initiated, payment.confirmed, payment.failed, payment.webhook_received

order-service	:8086	SYSTEM (no HTTP)	GET /health only	payment.confirmed	order.created, order.completed, refund.requested, loyalty.points_earned, notification.payment_receipt
exit-validation-service	:8087	CUSTOMER	POST /exit/validate, POST /exit/rfid-confirm	—	exit.validated, exit.denied, exit.override_used, exit.alarm_triggered
notification-service	:8088	SYSTEM (no HTTP)	GET /health only	notification.payment_receipt, notification.otp	—
loyalty-service	:8089	CUSTOMER	POST /loyalty/redeem, GET /loyalty/balance, GET /loyalty/history	loyalty.points_earned, order.completed	loyalty.tier_changed
inventory-service	:8090	STAFF	POST /inventory/grn, GET /inventory/low-stock, POST /inventory/adjust, GET /inventory/stock/{storeId}	warehouse.grn_confirmed	inventory.low_stock, inventory.shrinkage_detected, inventory.reorder_triggered
warehouse-service	:8091	STAFF	POST /warehouse/po, GET /warehouse/po/{id}, POST /warehouse/transfer	inventory.reorder_triggered	warehouse.grn_discrepancy, warehouse.grn_confirmed, warehouse.transfer_dispatched
analytics-service	:8092	CHAIN_HQ	GET /analytics/store/{id}, GET /analytics/chain/{id}, GET /analytics/sales	order.completed, cart.item_scanned, exit.validated	user.segment_updated
support-service	:8093	ALL	POST /support/ticket, GET /support/ticket/{id}, POST /support/ticket/{id}/message, PUT /support/ticket/{id}/status	—	support.ticket_created, support.ticket_escalated

retailer-auth-service	:8094	STAFF	POST /staff/otp/send, POST /staff/otp/verify, POST /staff/kiosk/session, POST /staff/logout	—	auth.staff_login
admin-auth-service	:8095	ZIPPYRA_ADMIN	POST /admin/login, POST /admin/2fa/verify, GET /admin/me, POST /admin/logout	—	admin.audit_event
chain-hq-service	:8096	CHAIN_HQ	GET /chain/{id}/stores, PUT /chain/{id}/config, POST /chain/erp/sync, GET /chain/{id}/reports	erp.sync_jobs	erp.sync_complete, erp.sync_failed, admin.config_changed
admin-store-service	:8097	ZIPPYRA_ADMIN	POST /admin/stores (10-step onboarding), POST /admin/stores/{id}/devices, GET /admin/stores/{id}/compliance	—	admin.store_onboarded, admin.config_changed, device.offline
compliance-service	:8098	SYSTEM (no HTTP)	GET /health only	account.deletion_request	admin.audit_event

3.2 The 7 Cross-System Dependencies

■ These 7 MUST be complete before any customer can use any pilot store. Run: `zippyra-admin verify-deps --store-id` to check all 7 atomically.

Dep	Name	Command	Tables / Keys Touched	Failure Mode if Missing
1	Store QR Tokens	zippyra-admin onboard	stores, admin_stores, store_qr_tokens, vendors, app_versions, Redis store_capacity:{id}	POST /store/bind returns 404 — no customers can enter the store
2	Product Catalogue	zippyra-admin catalog-import --file products.csv	products (PostgreSQL), sku:{barcode}:{store_id} (Redis catalog), product_availability:{id}:{store_id}	POST /cart/scan returns PRODUCT_NOT_FOUND for every barcode — self-checkout broken
3	JWT user_type enforcement	Built into shared/jwt + middleware	JWT claims, jwt_blacklist:{ti} in Redis sessions	STAFF tokens accepted on customer endpoints — security breach
4	RFID fallback mode	Automatic — checks devices table	devices WHERE device_type='RFID_PAD'	Exit service crashes if RFID check isn't skipped when no pad registered
5	HSN/GST Rate Table	zippyra-admin seed-hsn	hsn_gst_rates (PostgreSQL) — 80 pilot codes, 12,000+ production	All invoices show 0% GST — GSTN compliance violation, financial reporting wrong

6	Payment Webhook Endpoint	Deploy payment-service behind public ALB + IP allowlist	payment_webhook_events, payments.status, Redis gateway_circuit	Razorpay retries never received — orders never created after payment — customer pays, gate stays closed
7	Default Offer Ruleset	zippyra-admin seed-offer-rules --store-id	offer_rules:{store_id} in Redis cart cluster	Cart Service panics on nil Redis response during offer evaluation — entire cart broken



Database Layer — Complete

4.1 PostgreSQL — All 46 Tables by Service Group

■ Source of truth: `zippyra_postgresql_final__2_.dbml`. 46 tables across 12 service groups. Migrations: 000001–000014.

GROUP 1: Auth Service

users	user_id, phone (unique), name, email, is_deleted, created_at, last_login
auth_events	Append-only. Forever retention. NEVER UPDATE.
admin_actions	Append-only audit. target_type + target_id for JOIN.
login_attempts	Per-phone rate limiting audit. Index: (phone, attempt_at).
email_verifications	Token hash (never raw). Link flow.
app_versions	Seeded by admin-cli. platform PK (ios/android). Force update flag.

GROUP 2: Store Service

stores	id, name, gstin, lat/lng, max_capacity (dep fix), is_open, timezone
store_hours	(store_id, day_of_week) unique. is_holiday flag.
store_qr_tokens	token_hash (SHA-256). is_used = one-time. expires_at.
beacon_verifications	BLE/QR audit trail for dispute resolution.
aisle_layout	SVG store map for AR navigation (Phase 2).
shelf_positions	(store_id, sku_id) → AR overlay coords.

GROUP 3: Catalog Service

categories	Self-referencing parent_id. product_count denormalized.
products	barcode+store_id unique. brand (dep fix). sold_by_weight. sync_seq trigger.
pricing_rules	Effective price overrides with time window.
offers	Offer campaigns (separate from offer_rules).
offer_rules	Per-store compiled rules: BOGO, PERCENTAGE, FLAT, BUNDLE.
combo_rules / combo_items	Multi-product combo discounts.
recipes / recipe_ingredients	Bundle suggestions based on recipes.
planogram_map	Shelf compliance monitoring (Phase 2).

GROUP 4: Cart Service

coupons	Coupon code definitions, usage limits.
coupon_usage	(coupon_code, user_id) unique. Prevents double-use.
hsn_gst_rates	hsn_code unique. cgst+sgst+igst rates. effective_from date. Dep 5.

GROUP 5: Inventory Service

stock	(store_id, product_id) unique. available_qty CHECK >= 0. reserved_qty.
--------------	--

GROUP 6: Order Service

orders	BRIN index on created_at. invoice_number, invoice_s3_key.
order_items	IMMUTABLE after insert. NO UPDATE/DELETE rules applied. GSTN compliance.
refunds	Refund amount, gateway_refund_id, status.
refund_requests	User-facing refund request. approval workflow link.
approval_workflow	Manager approval for refunds > threshold.

GROUP 7: Payment Service

payments	idempotency_key UNIQUE. retry_count. gateway_order_id. gateway_payment_id.
payment_webhook_events	(gateway, gateway_event_id) UNIQUE. Dep 6. Idempotent webhook.
tokenized_cards	Gateway token only. No raw card data. PCI-DSS compliant.
reconciliation_log	(store_id, date, gateway) unique. Daily settlement reconcile.
payment_audit_log	Append-only FOREVER. Separate from payment_analytics_events.

GROUP 8: Loyalty Service

loyalty_tiers	BRONZE/SILVER/GOLD/PLATINUM. points_multiplier.
loyalty_cards	(user_id) unique. points_balance CHECK >= 0.
loyalty_ledger	Every earn/redeem. balance_after for audit trail.
loyalty_tier_history	Tier change history with reason.

GROUP 9: Exit Validation Service

exit_tokens	jti UNIQUE. is_used flag. expires_at. Links order → user → store.
override_log	Staff manual gate override. Append-only. Compliance record.

GROUP 10: Warehouse Service

warehouse_stock	Buffer stock at warehouse level.
transfers	Cross-store transfer records. Referenced by inventory_movements.

GROUP 11: Notification Service

user_notification_prefs	Per-user channel preferences (WhatsApp, SMS, push, email).
notification_delivery_log	Delivery status per notification. gateway_msg_id.

GROUP 12: Support Service

tickets	ticket_number unique. SLA tracking.
ticket_messages	Append-only thread. sender_type: CUSTOMER/AGENT/SYSTEM.

4.2 Redis — 6 Clusters (Complete Key Registry)

■ Source of truth: redis_keys.go + redis_cluster_config.yaml. ClusterFor() in shared/redis routes every key to correct cluster.

redis-sessions

eviction: noeviction

cache.r7g.2xlarge × 8 shards × 1 replica

```
otp:{phone} – TTL 600s
account_locked:{phone} – TTL 1800s
resend_cooldown:{phone} – TTL 60s
session:{user_id} – TTL 86400s
payment_pin_hash:{user_id} – TTL 30 days
device_fingerprint:{hash} – TTL 30 days
jwt_blacklist:{jti} – TTL = token expiry
fcm_token:{user_id} – TTL 90 days
whatsapp_rate:{phone} – TTL 60s
```

redis-cart

eviction: noeviction

cache.r7g.4xlarge × 16 shards × 1 replica

```
cart:{user_id}:{store_id} – TTL 7200s
exit_preauth:{user_id} – TTL 7200s
temp_exit:{user_id} – TTL 1800s
checkout_lock:{user_id} – TTL 600s
payment_lock:{user_id} – TTL 300s
user_store_session:{user_id}:{store_id} – TTL 14400s
scan_rate:{user_id} – rate limit sliding window
```

redis-catalog

eviction: allkeys-lru

cache.r7g.2xlarge × 8 shards × 1 replica

```
sku:{barcode}:{store_id} – TTL 86400s (fast barcode lookup)
offer_rules:{store_id} – TTL 300s
store_catalog_version:{store_id} – TTL 86400s
product_availability:{product_id}:{store_id} – TTL 30s
offer_personalized:{user_id} – TTL 900s
search_autocomplete_cache:{query_hash} – TTL 300s
trending_skus:{store_id} – TTL 3600s
hsn_gst:{hsn_code} – TTL 86400s
flash_sale_inventory:{offer_id}:{sku_id} – allkeys-lru (NOTE: moved here from cart)
```

redis-payments

eviction: noeviction

cache.r7g.xlarge × 4 shards × 1 replica

```
gateway_health:{gateway} – TTL 30s
gateway_circuit:{gateway} – TTL 60s
rate_limit:{endpoint}:{ip} – TTL 60s
payment_pin_lock:{user_id} – TTL 300s
```

redis-realtime

eviction: volatile-lru

cache.r7g.xlarge × 4 shards × 1 replica

```
gate_status:{gate_id} – TTL 30s
store_capacity:{store_id} – persistent (no TTL)
virtual_queue:{store_id} – sorted set by join time
store_event_queue:{store_id} – TTL 3600s
gst_in_cache:{gst_in} – TTL 86400s
exit_used:{jti} – TTL 600s (one-time exit QR)
```

redis-analytics

eviction: allkeys-lru

cache.r7g.large × 2 shards × 1 replica

```
loyalty_balance_cache:{user_id} – TTL 300s
feature_flags:{store_id} – TTL 300s
ab_test_assignment:{user_id} – TTL 7 days
```

4.3 Kafka — All 64 Topics

■ Source of truth: kafka_topics.go. Partition key = store_id for ALL topics (not user_id). cart.item_scanned has 256 partitions (highest volume). Financial topics (payment.*, order.*) retain 30 days minimum.

auth (7 topics)	auth.otp_sent auth.login_success auth.login_failed auth.account_locked auth.session_created auth.session_ended auth.staff_login
account (1 topic)	account.deletion_requested ← ALL services consume this for DPDP PII purge cascade
store (4 topics)	store.session_started store.queue_joined store.qr_generated store.server_recovered store.capacity_alert store.going_offline
catalog (4 topics)	catalog.product_updated catalog.import_job catalog.bulk_import_complete catalog.price_updated
cart (5 topics)	cart.item_scanned ← 256 partitions, highest volume cart.item_removed cart.abandoned cart.checkout_initiated cart.item_added
payment (6 topics)	payment.initiated payment.confirmed ← triggers order creation payment.failed payment.split_initiated payment.webhook_received order.payment_requested
order (2 topics)	order.created order.completed ← triggers loyalty, analytics, notification
exit (4 topics)	exit.validated exit.denied exit.override_used ← 30d retention exit.alarm_triggered
refund (2 topics)	refund.requested refund.completed
inventory (5 topics)	inventory.low_stock inventory.shrinkage_detected inventory.hold_request inventory.shrinkage_alert inventory.reorder_triggered

warehouse (3 topics)	warehouse.grn_discrepancy warehouse.transfer_dispatched warehouse.grn_confirmed
transfer (2 topics)	transfer.dispatched transfer.confirmed
qc (1 topic)	qc.inspection_complete
loyalty (2 topics)	loyalty.points_earned loyalty.tier_changed
support (2 topics)	support.ticket_created support.ticket_escalated
user (2 topics)	user.segment_updated user.account_merged
device (3 topics)	device.offline device.alarm device.heartbeat_missed
admin (3 topics)	admin.store_onboarded admin.config_changed admin.audit_event ← ALL services produce, sole consumer: Audit
erp (3 topics)	erp.sync_complete erp.sync_failed erp.sync_jobs
offer (1 topic)	offer.applied

4.4 TimescaleDB — 8 Hypertables + 20 Continuous Aggregates

Hypertable	Chunk Interval	Purpose
inventory_movements	1 DAY (fixed from 1 WEEK)	Every stock change. segmentby=store_id. Compress after 7d.
rfid_scan_events	1 HOUR	RFID tag reads at gate. High freq. Phase 2.
cart_scan_events	1 HOUR	Every barcode scan. Feeds trending SKU cache.
payment_analytics_events	5 MIN	Payment gateway health. Feeds Redis gateway_health:{gw}.
store_traffic	15 MIN	Store occupancy. Feeds Redis store_capacity.
conveyor_rfid_events	1 HOUR	Phase 3 conveyor RFID. Anti-theft pipeline.
fraud_signals	1 HOUR	Fraud pattern signals. Feeds ClickHouse ml_fraud_features.
supply_chain_events	1 DAY	GRN, transfer, vendor events. Phase 2.
20 Continuous Aggregates	Various	inv_hourly_stock, rfid_gate_hourly, cart_scans_daily, payment_gateway_5min, traffic_15min, payment_daily_summary, etc.

4.5 ClickHouse — 9 Analytics Tables

Table	Purpose	Feed Source
user_purchase_patterns	RFM segmentation, basket analysis	order_items (Kafka → consumer)
sales_summary	Daily/hourly revenue per store, chain	TimescaleDB payment_daily_summary
peak_hour_heatmap	Foot traffic patterns per store	TimescaleDB traffic_hourly
shrinkage_signals	Inventory loss signals per SKU	TimescaleDB rfid_daily_mismatch
co_purchase_matrix	Product affinity for recommendations	order_items pairs analysis
user_behavior_sessions	Scan path, dwell time, conversion	cart_scan_events stream
demand_forecasting_features	ML features for smart replenishment	sales_summary + inventory_movements
dynamic_pricing_signals	Demand-based price suggestion signals	purchase_patterns + peak_hour
ml_fraud_features	Exit fraud ML feature store	TimescaleDB fraud_signals

4.6 Elasticsearch — 5 Indexes

Index	Owner	Shards	Purpose
es_index_products	Catalog Service (write), Customer App (read)	16 (routing: store_id)	Product search, category filter, barcode fallback, autocomplete
es_index_stores	Admin Store Service (write), Customer App (read)	8 (routing: chain_id)	Nearby store search, geolocation queries
orders_search	Order Service (write), Admin (read)	16 (routing: store_id)	Order search for admin panel. PII masked.
support_tickets	Support Service (write), Admin (read)	8	Ticket search by customer, store, keyword
users_search	Auth Service (write), Admin (read)	8	Admin user lookup. Phone masked.

4.7 WatermelonDB — 11 Tables (Schema v11)

■ Source of truth: watermelondb_schema.js. Schema version 11. Migrations v1→v4.

Table	Purpose	Key Fields
local_cart	Offline cart. synced flag prevents double-submit on reconnect	sku_id, store_id, qty, unit_price, gst_rate, hsn_code, synced, is_deleted
catalog_cache	Offline product catalog for barcode lookup	barcode (indexed), store_id, mrp, effective_price, gst_rate, hsn_code, stock_status, has_offer

offer_cache	Compiled offer rules for offline offer evaluation	store_id, version, rules JSON, compiled_at
scan_buffer	Barcode scans buffered when API call fails	barcode, store_id, retry_count, last_error
recent_orders_cache	Last 20 orders for offline order history	order_id, store_id, total_amount, status
store_session	Active store session state	store_id, jti, entered_at, catalog_version
store_session_cache	Cached store info (name, hours, address)	store_id, name, is_open, catalog_version
feature_flags_cache	Remote config for feature toggles	flag_name, value, fetched_at
loyalty_balance_cache	Offline loyalty points display	user_id, points_balance, tier_name
notification_inbox	Offline notifications queue	message, read_at, deep_link
user_profile_cache	Offline profile (name, phone, preferences)	user_id, phone, name, dietary_prefs



Customer Mobile App (React Native)

5.1 Customer App Folder Structure (Complete)

zippyra-customer-app/

■■■■ app.json

■■■■ eas.json

■■■■ tsconfig.json

■■■■ babel.config.js

■■■■ package.json

■■■■ src/

■ ■■■■ app/

■ ■ ■■■■ _layout.tsx

■ ■ ■■■■ (auth)/

■ ■ ■■■■ (auth)/_layout.tsx

■ ■ ■■■■ (auth)/index.tsx

■ ■ ■■■■ (auth)/otp.tsx

■ ■ ■■■■ (auth)/version-check.tsx

■ ■ ■■■■ (tabs)/

■ ■ ■■■■ (tabs)/_layout.tsx

■ ■ ■■■■ (tabs)/home.tsx

■ ■ ■■■■ (tabs)/scan.tsx

■ ■ ■■■■ (tabs)/cart.tsx

■ ■ ■■■■ (tabs)/orders.tsx

■ ■ ■■■■ (tabs)/profile.tsx

■ ■ ■■■■ store/

■ ■ ■■■■ store/scan-entrance.tsx

■ ■ ■■■■ store/bound.tsx

■ ■ ■■■■ store/closed.tsx

■ ■ ■■■■ store/full.tsx

■ ■ ■■■■ payment/

■ ■ ■■■■ payment/checkout.tsx

■ ■ ■■■■ payment/processing.tsx

■ ■ ■■■■ payment/success.tsx

■ ■ ■■■■ payment/failed.tsx

■ ■ ■■■■ exit/

■ ■ ■■■■ exit/qr.tsx

■ ■ ■■■■ exit/denied.tsx

■ ■ ■■■■ orders/[id].tsx

■ ■ ■■■■ support/new-ticket.tsx

■ ■■■■ components/

■ ■ ■■■■ ui/

■ ■ ■■■■ ui/Button.tsx

■ ■ ■■■■ ui/Card.tsx

■ ■ ■■■■ ui/Badge.tsx

■ ■ ■■■■ ui/Input.tsx

■ ■ ■■■■ ui/Modal.tsx

React Native (Expo SDK 51) + TypeScript

Bundle ID: com.zippyra.app, splash, icons, scheme: zippyra

Profiles: development (simulator), preview (internal), production (store)

strict: true, paths: {'@/*': ['./src/*']}

expo preset + module-resolver plugin for @ aliases

Key deps: expo-camera, react-native-vision-camera, razorpay-react-native, @nozbe/watermelondb, zustand, @tanstack/react-query, expo-secure-store, react-native-qrcode-svg, sentry-expo

Expo Router file-based navigation

Root layout: ThemeProvider, QueryClient, Zustand, Sentry init

No tab bar. No auth required.

Redirect to (tabs) if already logged in

Splash → phone number entry → sendOTP()

6-digit OTP input → verifyOTP() → store JWT → navigate (tabs)

Force update screen shown if app < min_version

Main app screens with bottom tab bar

Tab bar: Home, Scan (in-store only), Cart, Orders, Profile

Store map (nearby stores), current session banner, promotions

Barcode scanner screen. Only active when store JWT has store_id.

Cart items, offer banners, item totals, proceed to checkout

Order history list from recent_orders_cache + API

User profile, loyalty card, notification prefs, logout

Store entry flow

Camera → decode entrance QR → POST /store/bind → success

Store entry confirmed. Trigger catalog sync. Show store info.

STORE_CLOSED error screen with opening hours

STORE_FULL error screen with estimated wait

Payment flow screens

Cart summary with HSN-wise GST breakdown, total, pay button

Spinner + poll payment status every 2s, max 30s timeout

Receipt preview + exit QR display + download invoice

Payment failure with retry and support ticket option

Exit flow

Exit QR (react-native-qrcode-svg) + 10min countdown timer + GateAnimation

Gate denied screen with reason + support contact

Order detail: items, GST breakdown, invoice download link

Create support ticket for order issues

Reusable UI components

Design system

Primary/secondary/ghost variants, loading state, haptic feedback

Surface with shadow, border radius

Offer badge, loyalty tier badge

Phone input with +91 prefix, OTP input with auto-advance

Bottom sheet modal (react-native-bottom-sheet)

■ ■ ■■ ui/Toast.tsx	Success/error/info toasts
■ ■ ■■ scanner/	
■ ■ ■■ scanner/BarcodeScanner.tsx	react-native-vision-camera + frame processor. Decode EAN-13/QR in <50ms.
■ ■ ■■ scanner/ScanLine.tsx	Animated scan line overlay
■ ■ ■■ scanner/ProductCard.tsx	Product popup after scan: name, price, offer badge
■ ■ ■■ cart/	
■ ■ ■■ cart/CartItem.tsx	Item row: image, name, brand, price, qty stepper, remove
■ ■ ■■ cart/CartSummary.tsx	Subtotal, GST breakdown by rate, offer savings, total
■ ■ ■■ cart/OfferBanner.tsx	Horizontal scroll of active offers
■ ■ ■■ payment/	
■ ■ ■■ payment/PaymentSheet.tsx	Razorpay SDK wrapper. Opens UPI/card/netbanking native sheet.
■ ■ ■■ payment/GSTBreakdown.tsx	CGST + SGST per HSN rate group table for invoice display
■ ■ ■■ exit/	
■ ■ ■■ exit/ExitQRCode.tsx	QR code with store branding overlay
■ ■ ■■ exit/GateAnimation.tsx	Gate open Lottie animation on exit success
■ ■ ■■ loyalty/LoyaltyCard.tsx	Loyalty tier badge, points balance, tier progress bar
■ ■ ■■ store/StoreCard.tsx	Store list item: name, distance, open/closed status
■ ■■■ db/	WatermelonDB offline database
■ ■ ■■ schema.ts	appSchema v11 — 11 tables (see Part 4.7)
■ ■ ■■ migrations/	
■ ■ ■■ migrations/index.ts	schemaMigrations: v1→v2→v3→v4 (addColumnns on catalog_cache)
■ ■ ■■ models/	WatermelonDB Model classes
■ ■ ■■ models/LocalCart.ts	extends Model, @field decorators for all cart fields
■ ■ ■■ models/CatalogCache.ts	extends Model, @field decorators, @readonly id
■ ■ ■■ models/OfferCache.ts	extends Model, parseRules() helper
■ ■ ■■ models/StoreSession.ts	extends Model, isExpired() computed
■ ■ ■■ models/RecentOrder.ts	extends Model
■ ■ ■■ sync.ts	pullChanges(lastPulledAt) + pushChanges(changes). ETag conditional. Jitter.
■ ■■■ hooks/	
■ ■ ■■ useAuth.ts	isLoggedIn, userId, phone, login(), logout(), refreshToken()
■ ■ ■■ useCart.ts	items, totals, scanBarcode(), removeItem(), clearCart()
■ ■ ■■ useSync.ts	triggerSync(storeId, catalogVersion) with ±30s jitter
■ ■ ■■ useStore.ts	activeStore, storeId, bindStore(), exitStore()
■ ■ ■■ useOffers.ts	offers, shouldRefresh (ageMs > 300000), loadOffers()
■ ■ ■■ usePayment.ts	initiatePayment(), pollStatus(), exitToken
■ ■ ■■ useLoyalty.ts	balance, tier, history, redeemPoints()
■ ■ ■■ usePermissions.ts	Camera, location, notifications permissions
■ ■■■ services/	All HTTP calls — never call API directly from components
■ ■ ■■ api.ts	Axios instance: baseUrl, JWT interceptor (auto-attach + auto-refresh), request ID header
■ ■ ■■ auth.service.ts	sendOTP(phone), verifyOTP(phone, otp), refreshToken(), logout(), getMe(), versionCheck()
■ ■ ■■ store.service.ts	bindStore(qrToken), exitStore(), getStoreInfo(id), getStoreHours(id)
■ ■ ■■ cart.service.ts	scanBarcode(barcode), removeItem(productId), getCart(), clearCart()
■ ■ ■■ payment.service.ts	initiatePayment(idempotencyKey, cartId, gateway), getPaymentStatus(id)
■ ■ ■■ catalog.service.ts	searchProducts(query, filters), getProduct(barcode), syncCatalog(storeId, since)
■ ■ ■■ loyalty.service.ts	getBalance(), getHistory(), redeemPoints(amount)

■ ■ ■■ support.service.ts	<i>createTicket(orderId, subject, description), getTickets()</i>
■ ■■■ store/	<i>Global state (Zustand)</i>
■ ■ ■■ auth.slice.ts	<i>{ accessToken, refreshToken, userId, phone, userType, isLoggedIn }</i>
■ ■ ■■ cart.slice.ts	<i>{ items[], totals, cartId, lastSyncedAt }</i>
■ ■ ■■ session.slice.ts	<i>{ storeId, storeName, catalogVersion, enteredAt, exitToken }</i>
■ ■ ■■ ui.slice.ts	<i>{ isLoading, toastMessage, activeModal }</i>
■ ■■■ utils/	
■ ■ ■■ barcode.ts	<i>validateEAN13(barcode), detectBarcodeType(), formatBarcode()</i>
■ ■ ■■ price.ts	<i>formatINR(paise), calculateGST(price, rate), roundToNearestPaise()</i>
■ ■ ■■ jwt.ts	<i>decodeJWT(token), isExpired(token), extractClaims(token)</i>
	<i>secureGet/Set/Delete via expo-secure-store. Never plain AsyncStorage for tokens.</i>
■ ■ ■■ storage.ts	<i>generateIdempotencyKey() → UUID v4 persisted in secure storage per cart</i>
■ ■ ■■ idempotency.ts	
■ ■ ■■ time.ts	<i>formatTime(), countdown(expiresAt), jitter(base, rangeMs)</i>
■ ■■■ constants/	
■ ■ ■■ api.ts	<i>BASE_URL per env (dev/staging/prod), endpoint constants</i>
■ ■ ■■ colors.ts	<i>Zippyra design tokens — primary, accent, background, text</i>
■ ■ ■■ typography.ts	<i>Font sizes, weights, line heights</i>
■ ■ ■■ config.ts	<i>Feature flags, timeout values, retry config</i>
■ ■■■ assets/	
■ ■ ■■ icons/	<i>App icon (1024×1024), adaptive icon, favicon</i>
■ ■ ■■ splash/	<i>Splash screen (2048×2048 + background color)</i>
■ ■ ■■ fonts/	<i>Inter variable font</i>
■ ■ ■■ animations/	<i>Lottie: gate-open.json, success-checkmark.json, loading-scan.json</i>

5.2 Core Screens & User Journey

Screen	Route	Key Actions
Splash / Phone Entry	(auth)/index	Send OTP, version check on mount
OTP Verify	(auth)/otp	Auto-submit on 6th digit, countdown resend timer
Home	(tabs)/home	Nearby stores map, scan entrance QR button when near store
Entrance QR Scan	store/scan-entrance	Camera → QR decode → bindStore() → catalog sync trigger
Scan Products	(tabs)/scan	Continuous scan mode, product popup, add to cart
Cart	(tabs)/cart	Live totals, offer banners, remove items, checkout button
Checkout	payment/checkout	GST breakdown, loyalty discount, Razorpay pay button
Payment Processing	payment/processing	Poll every 2s, max 30s, then show retry/support
Payment Success	payment/success	Receipt, exit QR prominently, download invoice
Exit QR	exit/qr	Full-screen QR + 10min countdown, help button
Orders	(tabs)/orders	History from WatermelonDB cache + API, tap for detail
Profile	(tabs)/profile	Loyalty card, points, tier progress, notification prefs, logout



Retailer Dashboard + Staff App + Electron Kiosk

6.1 Retailer Dashboard Folder Structure (Complete)

zippyra-retailer-dashboard/	Next.js 14 + TypeScript (retailer.zippyra.com)
■■■ next.config.ts	PWA config (next-pwa), image domains (CDN), API rewrites to backend
■■■ tailwind.config.ts	Zippyra design tokens extended from shadcn/ui
■■■ middleware.ts	NextAuth session check, redirect unauthenticated → /login
■■■ src/	
■ ■■■ app/	Next.js 14 App Router
■ ■ ■■ (auth)/	
■ ■ ■■ (auth)/login/page.tsx	Staff phone + OTP login form → POST /v1/staff/otp/verify
■ ■ ■■ (auth)/logout/page.tsx	Clear session, redirect /login
■ ■ ■■ dashboard/	Protected — requires STAFF or CHAIN_HQ JWT
■ ■ ■■ dashboard/layout.tsx	Sidebar + TopNav shell. Real-time alert badge (WebSocket).
■ ■ ■■ dashboard/page.tsx	Store KPIs: today's sales, active customers, low stock count
■ ■ ■■ dashboard/inventory/	
■ ■ ■■ dashboard/inventory/page.tsx	Stock table: product, available_qty, reserved_qty, reorder_point. Filters.
■ ■ ■■ dashboard/inventory/grn/	
■ ■ ■■ dashboard/inventory/grn/page.tsx	GRN list. Create new GRN from PO.
■ ■ ■■ dashboard/inventory/grn/[id]/	
■ ■ ■■ dashboard/inventory/grn/[id]/page.tsx	GRN receive: scan barcodes, enter qty, accept/reject items
■ ■ ■■ dashboard/inventory/adjustments/page.tsx	Manual stock adjustment with reason and approval
■ ■ ■■ dashboard/orders/	
■ ■ ■■ dashboard/orders/page.tsx	All orders. Filter by date, status, amount. Refund initiation.
■ ■ ■■ dashboard/orders/[id]/page.tsx	Order detail: items, GST breakdown, payment status, refund button
■ ■ ■■ dashboard/analytics/	
■ ■ ■■ dashboard/analytics/page.tsx	ClickHouse-powered: sales chart, peak hours heatmap, top SKUs
■ ■ ■■ dashboard/analytics/inventory/	Shrinkage signals, stock health, reorder recommendations
■ ■ ■■ dashboard/staff/	
■ ■ ■■ dashboard/staff/page.tsx	Staff list, roles, active shifts
■ ■ ■■ dashboard/staff/[id]/page.tsx	Staff profile, role edit, deactivate
■ ■ ■■ dashboard/devices/	
■ ■ ■■ dashboard/devices/page.tsx	Device registry: status, last heartbeat, firmware version
■ ■ ■■ dashboard/devices/[id]/page.tsx	Device detail: events log, config, restart/update
■ ■ ■■ dashboard/offers/	
■ ■ ■■ dashboard/offers/page.tsx	Active/scheduled offers. Create new offer rule.
■ ■ ■■ dashboard/offers/builder/page.tsx	Visual offer builder: BOGO, PERCENTAGE, BUNDLE, COMBO
■ ■ ■■ dashboard/catalog/	
■ ■ ■■ dashboard/catalog/page.tsx	Product list, search, bulk price edit
■ ■ ■■ dashboard/catalog/import/page.tsx	CSV drag-drop upload → catalog-import endpoint
■ ■ ■■ dashboard/catalog/[id]/page.tsx	Product edit: name, brand, price, HSN, image upload
■ ■ ■■ dashboard/support/	
■ ■ ■■ dashboard/support/page.tsx	Customer support tickets assigned to this store
■ ■ ■■ api/	Next.js API routes (BFF — Backend For Frontend)
■ ■ ■■ api/auth/[...nextauth]/route.ts	NextAuth with credentials provider (staff JWT)
■ ■ ■■ api/analytics/[...]/route.ts	Proxy to analytics-service with CHAIN_HQ JWT
■ ■■■ components/	
■ ■ ■■ charts/	
■ ■ ■■ charts/SalesChart.tsx	Recharts AreaChart: hourly/daily/weekly sales

■ ■ ■■ charts/InventoryHeatmap.tsx	Stock level heat map by category
■ ■ ■■ charts/PeakHoursChart.tsx	ClickHouse peak_hour_heatmap bar chart
■ ■ ■■ charts/ShrinkageChart.tsx	Shrinkage signals over time
■ ■ ■■ tables/	
■ ■ ■■ tables/DataTable.tsx	TanStack Table: sort, filter, pagination, column visibility
■ ■ ■■ tables/StockTable.tsx	Stock with color-coded low stock rows
■ ■ ■■ tables/OrdersTable.tsx	Orders with status badge, refund action
■ ■ ■■ forms/	
■ ■ ■■ forms/GRNForm.tsx	Receive goods: barcode scan input, qty comparison, notes
■ ■ ■■ forms/ProductForm.tsx	Add/edit product: all fields including HSN code lookup
■ ■ ■■ forms/OfferBuilder.tsx	Step-by-step offer rule builder with preview
■ ■ ■■ layout/	
■ ■ ■■ layout/Sidebar.tsx	Role-aware navigation: MANAGER sees everything, CASHIER sees orders only
■ ■ ■■ layout/TopNav.tsx	Store switcher, alert bell (real-time count), user menu
■ ■ ■■ layout/AlertBanner.tsx	Real-time low stock / device offline alerts via SSE
■ ■■■ lib/	
■ ■ ■■ api.ts	Axios with staff JWT auto-attach from NextAuth session
■ ■ ■■ clickhouse.ts	ClickHouse HTTP client for direct analytics queries
■ ■ ■■ sse.ts	Server-Sent Events client for real-time alerts
■ ■ ■■ auth.ts	NextAuth config: credentials provider → POST /v1/staff/otp/verify
■ ■■■ hooks/	
■ ■ ■■ useInventory.ts	Stock data, low stock alerts, GRN list
■ ■ ■■ useOrders.ts	Orders with filters, refund actions
■ ■ ■■ useAnalytics.ts	ClickHouse data hooks with SWR
■ ■ ■■ useDevices.ts	Device status, heartbeat subscription
■ ■ ■■ useRealTimeAlerts.ts	SSE subscription for low stock + device events
■ ■■■ types/	TypeScript interfaces for all API responses

6.2 Staff App Folder Structure (Complete)

zippyra-staff-app/	React Native (Expo) — TypeScript
■■■ app.json	Bundle ID: com.zippyra.staff
■■■ src/	
■ ■■ app/	Expo Router
■ ■ ■■ (auth)/login.tsx	Staff phone + OTP → STAFF JWT
■ ■ ■■ (auth)/select-role.tsx	If staff has multiple store assignments, select store
■ ■ ■■ (tabs)/home.tsx	Shift start/end, active alerts, quick action shortcuts
■ ■ ■■ (tabs)/inventory.tsx	Quick stock count: scan barcode → show current qty → enter counted qty
■ ■ ■■ (tabs)/grn.tsx	Mobile GRN receive: scan items, enter qty, confirm receipt
■ ■ ■■ (tabs)/orders.tsx	Today's orders, customer lookup by phone, assist checkout
■ ■ ■■ (tabs)/alerts.tsx	Low stock alerts, device alarms, gate alarms
■ ■ ■■ devices/rfid.tsx	RFID pad deactivation screen for post-exit cleanup
■ ■ ■■ devices/gate.tsx	Gate status, manual override with reason entry
■ ■ ■■ customer/assist.tsx	Customer assistance: barcode lookup, price check, availability
■ ■■■ components/	Staff-specific UI
■ ■ ■■ AlertCard.tsx	Actionable alert card: low stock with reorder button
■ ■ ■■ StockCountRow.tsx	Scan + count interface for cycle counting
■ ■ ■■ GRNReceiveRow.tsx	Expected vs received qty comparison

■ ■■■ services/	
■ ■ ■■■ staff-api.ts	<i>Axios with STAFF JWT from secure storage</i>
■ ■ ■■■ inventory.service.ts	<i>getStock(), submitGRN(), adjustStock()</i>
■ ■ ■■■ device.service.ts	<i>getDeviceStatus(), triggerRFIDDeactivation()</i>
■ ■■■ hooks/	<i>useShift, useInventoryCount, useAlerts, useGate</i>

6.3 Electron Kiosk Folder Structure

zippyra-kiosk/	<i>Electron + React + TypeScript (in-store self-service terminal)</i>
■■■ electron/	
■ ■■■ main.ts	<i>Electron main process: kiosk mode (fullscreen, no menu bar)</i>
■ ■■■ preload.ts	<i>Expose barcode scanner HID events to renderer</i>
■ ■■■ hardware.ts	<i>USB barcode scanner, receipt printer, touchscreen config</i>
■■■ src/	<i>React renderer</i>
■ ■■■ screens/	
■ ■ ■■■ Idle.tsx	<i>Attract screen, language select</i>
■ ■ ■■■ PhoneEntry.tsx	<i>Customer phone number entry (touchscreen numpad)</i>
■ ■ ■■■ OTPEntry.tsx	<i>OTP verification</i>
■ ■ ■■■ Scan.tsx	<i>Barcode scan + cart (same as mobile scan+cart flow)</i>
■ ■ ■■■ Payment.tsx	<i>Razorpay payment terminal integration</i>
■ ■ ■■■ Receipt.tsx	<i>Print receipt + display exit QR</i>
■ ■■■ services/	<i>Same API services as customer app</i>



Admin & Chain HQ Platform

7.1 Admin Platform Folder Structure (Complete)

zippyra-admin-platform/	Next.js 14 monorepo — Turborepo
■■■■ turbo.json	Pipeline: build, lint, test, dev
■■■■ package.json	Workspaces: apps/admin, apps/hq, packages/*
■■■■ apps/	
■ ■■■■ admin/	admin.zippyra.com — Zippyra internal team
■ ■ ■■■■ src/app/	
■ ■ ■■■■ src/app/stores/	Store list, onboarding wizard
	10-step wizard: Details→Geofence→Hardware→Staff→Catalog→HSN→Payment→Offers→QR→Go-Live
■ ■ ■■■■ src/app/stores/new/	Store health dashboard
■ ■ ■■■■ src/app/stores/[id]/overview/	Device registry: 8 types. Heartbeat monitor. Register via MQTT.
■ ■ ■■■■ src/app/stores/[id]/devices/	DPDP status, PCI-DSS scan, GSTN reconciliation
■ ■ ■■■■ src/app/stores/[id]/compliance/	Generate + print QR token batches
■ ■ ■■■■ src/app/stores/[id]/qr-tokens/	Chain list, create chain, subscription management
■ ■ ■■■■ src/app/chains/	All stores in chain with KPI rollout
■ ■ ■■■■ src/app/chains/[id]/stores/	Customer search, account status, DPDP deletion queue
■ ■ ■■■■ src/app/users/	
■ ■ ■■■■ src/app/system/	
■ ■ ■■■■ src/app/system/kafka/	Kafka DLQ viewer: depth per topic, replay/discard
■ ■ ■■■■ src/app/system/circuit-breakers/	Payment gateway circuit breaker status panel
■ ■ ■■■■ src/app/system/feature-flags/	Remote config flag management
■ ■ ■■■■ src/app/audit/	admin_actions audit trail: filter by admin, target, action type
■ ■ ■■■■ src/app/hsn/	HSN/GST rate management: seed, update, full import
■ ■■■■ hq/	hq.zippyra.com — Chain HQ users (Big Bazaar, Reliance, etc.)
■ ■ ■■■■ src/app/dashboard/	Multi-store KPI overview: revenue, active customers, top stores
■ ■ ■■■■ src/app/stores/	Store-level drill-down (read-only for Chain HQ)
■ ■ ■■■■ src/app/analytics/	ClickHouse: cross-store sales, inventory, shrinkage, demand forecast
■ ■ ■■■■ src/app/catalog/	Chain-wide catalog: bulk price update, product add/remove across stores
■ ■ ■■■■ src/app/offers/	Chain-wide offer campaigns: create, schedule, A/B test
■ ■ ■■■■ src/app/staff/	Chain-wide staff management, role assignment
■ ■ ■■■■ src/app/erp/	ERP integration config: SAP, Tally, Busy. Sync status. Error log.
■ ■ ■■■■ src/app/reports/	Downloadable reports: sales (PDF/Excel), GST reconciliation, inventory
■■■■ packages/	Shared across admin + hq
■ ■■■■ ui/	shadcn/ui + Zippyra design tokens. Storybook documented.
	Type-safe fetch wrapper for all 18 service APIs. Auto-generated from OpenAPI.
■ ■■■■ api-client/	NextAuth: ZIPPYRA_ADMIN (TOTP 2FA) + CHAIN_HQ (SSO) providers
■ ■■■■ auth/	useStores(), useChain(), useCompliance(), useKafka(), useFeatureFlags()
■ ■■■■ hooks/	
■ ■■■■ types/	Shared TypeScript interfaces (Store, Chain, Device, Ticket, etc.)
■ ■■■■ charts/	ClickHouse-powered chart components shared by admin + hq

7.2 Store Onboarding — 10-Step Wizard API

Step	Admin UI Action	API Call	DB / Redis Change
------	-----------------	----------	-------------------

1 Store Details	Name, address, GSTIN, city, state	POST /admin/stores {name, gstin, lat, lng}	INSERT stores, INSERT admin_stores (status=DRAFT)
2 Geofence	Draw polygon on Mapbox	PUT /admin/stores/{id}/geofence {polygon}	UPDATE admin_stores.geofence_polygon
3 Hardware Config	Register each device	POST /admin/stores/{id}/devices {type, serial}	INSERT devices. Publishes MQTT message to device.
4 Staff Onboarding	Add staff, assign roles	POST /admin/stores/{id}/staff {phone, role}	INSERT staff_members. staff JWT issued on first login.
5 Catalog Import	CSV upload	POST /v1/admin/catalog/import {store_id, csv}	INSERT products (bulk), SET sku:{barcode}:{store_id} Redis
6 HSN/GST Verify	Check all products have valid HSN	GET /admin/stores/{id}/hsn-coverage	JOIN products + hsn_gst_rates. Returns unmatched list.
7 Payment Setup	Enter Razorpay credentials	PUT /admin/stores/{id}/payment-config	Store in AWS Secrets Manager. Test webhook call.
8 Offer Rules	Create first offer or skip	POST /admin/stores/{id}/offers (optional)	INSERT offer_rules. SET offer_rules:{store_id} Redis.
9 QR Generation	Generate + download QR PDF	POST /admin/stores/{id}/qr-tokens {count}	INSERT store_qr_tokens. Return signed QR payloads for print.
10 Go-Live Check	Run all dependency checks	POST /admin/stores/{id}/verify-deps	Runs all 7 cross-system dep checks. Status: READY or BLOCKED.



Infrastructure & DevOps

8.1 Terraform Module Structure

infra/	Terraform root
■ ■ ■ modules/	
■ ■ ■ ■ vpc/	VPC, public subnets (ALB), private subnets (EKS), db subnets (RDS/Redis), NAT GW, IGW, Security Groups
■ ■ ■ ■ rds/	PostgreSQL 16 RDS Multi-AZ, parameter group (max_connections=200), subnet group, 7d backup, Performance Insights
■ ■ ■ ■ elasticache/	6 Redis clusters, per-cluster: node type, eviction policy, shard count, auth token, encryption at rest
■ ■ ■ ■ msk/	Kafka MSK: broker count, kafka.t3.small pilot / kafka.m5.xlarge prod, SASL/SCRAM auth, JMX metrics
■ ■ ■ ■ eks/	EKS 1.30: managed node groups, IRSA for service accounts, cluster autoscaler, AWS Load Balancer Controller
■ ■ ■ ■ iam/	IRSA roles per service (least privilege), KMS keys for Secrets Manager
■ ■ ■ ■ secrets/	Secrets Manager: DB password, Redis auth, Kafka SASL, JWT private key, Razorpay keys, Twilio keys
■ ■ ■ ■ s3/	Buckets: zippyra-invoices (private, lifecycle 90d→glacier), zippyra-products (private), zippyra-media (CloudFront)
■ ■ ■ ■ opensearch/	AWS OpenSearch: domain, 5 indexes, ILM policies (delete after 90d for logs)
■ ■ ■ ■ iot/	AWS IoT Core: thing type per device type (8 types), MQTT topics, certificate provisioning, device shadow
■ ■ ■ ■ cloudfront/	CDN for zippyra-media S3 bucket. Product images, store media.
■ ■ ■ environments/	
■ ■ ■ ■ pilot/	5 stores, ~50 concurrent users
■ ■ ■ ■ main.tf	Call all modules with pilot vars
■ ■ ■ ■ variables.tf	
■ ■ ■ ■ terraform.tfvars	db_instance=t4g.large, kafka_count=3, eks_nodes=3, redis_node=t4g.medium
■ ■ ■ ■ outputs.tf	EKS endpoint, RDS endpoint, MSK brokers, Redis endpoints
■ ■ ■ ■ production/	100M customer scale
■ ■ ■ ■ main.tf	
■ ■ ■ ■ variables.tf	
■ ■ ■ ■ terraform.tfvars	db_instance=r6g.2xlarge, kafka_count=12, eks_min=6, redis_node=r7g.xlarge (gitignored)
■ ■ ■ ■ outputs.tf	

8.2 Pilot vs Production Sizing

Component	Pilot	Phase 1 (10k stores)	Production (1M+)
PostgreSQL	db.t4g.large Multi-AZ + 2 read replicas	db.r6g.2xlarge + 5 read replicas + RDS Proxy	Citus sharding, shard key: store_id
Redis Sessions	cache.t4g.medium ×2, no cluster	cache.r6g.large, 4 shards × 1 replica	cache.r7g.2xlarge, 8 shards × 1 replica
Redis Cart	cache.t4g.medium ×2	cache.r6g.xlarge, 8 shards × 1 replica	cache.r7g.4xlarge, 16 shards × 1 replica
Kafka (MSK)	3 × kafka.t3.small, 1 AZ	6 × kafka.m5.large, multi-AZ	12 × kafka.m5.xlarge + dedicated payment cluster

EKS Nodes	3 × t3.medium	6 × t3.large, spot+on-demand mix	Auto-scaling groups: t3.large→c5.2xlarge
TimescaleDB	Timescale Cloud Dev (8 GB)	Timescale Cloud Pro (32 GB)	Timescale Cloud Business (dedicated)
ClickHouse	ClickHouse Cloud Dev	ClickHouse Cloud Business	ClickHouse Cloud Dedicated
OpenSearch	t3.small.search ×1	r6g.large.search ×3 (replicated)	r6g.2xlarge.search ×5
Apply order	VPC→IAM→RDS+Redis(parallel)→MSK→EKS (~45 min)	Same	Blue/green deploy, no full re-apply

8.3 Kubernetes Configuration

Aspect	Config	Why
Namespaces	zippyra-pilot, zippyra-prod, monitoring, kafka	Hard isolation. Prod can't touch pilot namespace.
Replicas	auth: 2min. payment: 3min. order/exit: 2min. others: 2min.	payment needs 3 for 99.99% uptime target.
HPA	CPU > 70% → scale. Min 2, Max 10 per service.	All services are stateless — safe to autoscale.
Resource requests	CPU 200m/500m, Memory 256Mi/512Mi	Right-sized for Go services. Adjust post-load-test.
Liveness probe	GET /health, initialDelay=10, period=15	Restart pod if deadlocked.
Readiness probe	GET /health, initialDelay=5, period=5	Remove from load balancer during startup.
Rolling update	maxSurge=1, maxUnavailable=0	Zero downtime. New pod ready before old terminates.
Pod disruption budget	minAvailable=1 for all services	At least 1 pod running during node maintenance.
Ingress	AWS ALB, SSL at ALB, HTTP→HTTPS redirect	No SSL termination at pod — reduces CPU.
Secrets	Kubernetes Secrets from AWS Secrets Manager via CSI driver	Secrets injected as env vars, auto-rotate.
Service mesh	Phase 2: Istio for mTLS between services	Pilot: no Istio. Add when microservice count grows.

8.4 CI/CD — 4 GitHub Actions Workflows

Workflow	Trigger	Steps
lint-and-test.yml	Every PR	1. go vet ./... 2. golangci-lint run 3. docker-compose up -d (postgres+redis+kafka+opensearch) 4. go test ./... -coverprofile 5. coverage ≥70% gate on auth, payment, exit-validation 6. docker-compose down
deploy-staging.yml	Merge to main	1. Build Docker image per changed service (affected by diff) 2. Push to ECR 3. kubectl rollout restart -n zippyra-pilot 4. Wait for rollout complete 5. Run smoke tests (10 synthetic transactions) 6. Notify Slack

deploy-production.yml	Manual + 1 approver	1-3. Same as staging but -n zippyra-prod 4. Monitor error rate for 5 min post-deploy 5. If error rate >1%: kubectl rollout undo 6. Publish deploy event to PagerDuty
eas-build.yml	git tag v* push	1. eas build --platform android --profile production 2. eas build --platform ios --profile production 3. Wait for builds 4. eas submit -p android (Google Play internal track) 5. eas submit -p ios (TestFlight)

8.5 Observability Stack

Component	Tool	Config
Metrics	Prometheus (kube-prometheus-stack)	Every Go service: prometheus/client_golang. Custom metrics: cart_scan_duration_ms, payment_gateway_error_rate, exit_validation_result_total.
Distributed Tracing	OpenTelemetry + Jaeger	otel.Init(serviceName) in every main.go. OTLP exporter → Jaeger. Full trace: scan→pay→exit shows all spans across 5 services.
Logging	zerolog → Fluent Bit → CloudWatch	Structured JSON. Level: INFO prod, DEBUG staging. Log: request_id, user_id, store_id, duration_ms.
Dashboards	Grafana (3 pre-pilot)	1. Pilot Store Health: capacity, scan rate, cart conversion 2. Payment Gateway Health: error rate, circuit state, P99 latency 3. Core Flow Funnel: scan→pay→exit conversion per store
Alerting	AlertManager → PagerDuty	Alerts: error_rate >1% (P1), P99_latency >500ms (P2), DLQ_depth >0 (P2), gateway_circuit_open (P1), pod_crashloopbackoff (P1)
Kafka DLQ	Custom Grafana panel	DLQ depth per topic. Alert fires within 30s of first DLQ message.
Synthetic Monitoring	Grafana Cloud	POST /v1/auth/otp/send every 60s from Mumbai + Singapore + US-East. Alert if >2 consecutive fails.
Error Tracking	Sentry (mobile)	sentry-expo in customer app. Crash reports with breadcrumbs.



18-Week Execution Roadmap

9.1 Phase 0 — Foundation (Weeks 1–4)

Week 1: Infrastructure + CI/CD + Shared Packages

- Terraform apply: VPC → IAM → RDS+Redis (parallel) → MSK → EKS. Total ~45 min.
- GitHub Actions: 4 workflows created. Secrets configured in GitHub.
- shared/db: pgxpool factory with 3-retry, 5s timeout, slow query tracer
- shared/redis: Clients{} with 6 clusters, ClusterFor() routing, all 4 Lua scripts
- shared/kafka: Producer (store_id partition enforced), Consumer (DLQ 3-retry), 64 topic constants
- shared/jwt: Ed25519 sign/verify, Claims struct, Revoke(), user_type enforcement
- shared/middleware: Standard(), JWTAuth(), CustomerOnly(), StoreScopeRequired()
- shared/otel: otel.Init(serviceName) called in every service main.go
- docker-compose.yml: postgres + 6 redis + kafka + zookeeper + opensearch + jaeger
- Run all 14 migrations: 000001 → 000014

Week 2: Observability + Admin CLI + ED25519 Keys

- kube-prometheus-stack + opentelemetry-collector + Jaeger deployed to monitoring namespace
- 3 Grafana dashboards configured and showing data from health endpoints
- ED25519 key pair generated: openssl genpkey -algorithm ed25519 → store in Secrets Manager
- zippyra-admin CLI compiled: onboard, catalog-import, seed-hsn, seed-offer-rules, verify-deps
- Pilot store 1 onboarded: zippyra-admin onboard --name 'Big Bazaar Koramangala' --lat 12.93 --lng 77.62 --gstn 29XXXXX
- HSN codes seeded: zippyra-admin seed-hsn (pilot 80 codes, trigger full GSTN import)
- offer_rules seeded: zippyra-admin seed-offer-rules --store-id
- All 7 deps verified: zippyra-admin verify-deps --store-id → ALL PASS

Week 3: Auth Service (Complete — all 10 steps each)

- POST /v1/auth/otp/send: phone validation, OTPRateLimitScript (Lua), generate+hash OTP, Twilio async send
- POST /v1/auth/otp/verify: hash compare, account lock on 5th fail, user upsert, JWT issue, session write
- POST /v1/auth/refresh: verify refresh token, revoke old, issue new pair
- POST /v1/auth/logout: revoke JWT (add to blacklist), delete session
- GET /v1/auth/version-check: reads app_versions. Force update if below min_version.
- Integration test: OTP flow end-to-end with real Redis and PostgreSQL (Docker Compose)

Week 4: Store Service + Catalog Service

- POST /v1/store/bind: parseQRToken → SHA-256 hash → DB lookup → is_used check → expiry → StoreCapacityIncrementScript → JWT upgrade → session write → Kafka
- GET /v1/store/{id}/info, GET /v1/store/{id}/hours, POST /v1/store/exit
- GET /v1/catalog/barcode/{barcode}: Redis cache hit → PostgreSQL read replica fallback → write-back
- GET /v1/catalog/search: Elasticsearch query with filters (category, brand, price range)

- POST /v1/catalog/sync: delta sync via sync_seq, ETag conditional (304 if up to date)

9.2 Phase 0 — Core Flow (Weeks 5–12)

Week 5: Cart Service (Target: <100ms P99)

- POST /v1/cart/scan: Redis SKU lookup → InventoryHoldScript (Lua atomic) → cart load → item add → evaluateOffers() → recalculate → save
- evaluateOffers(): pure Go, no I/O, reads pre-loaded offer_rules JSON. Handles PERCENTAGE, BOGO, BUNDLE, COMBO.
- runInventoryReconciler(): background goroutine, 10s ticker, Redis holds → PostgreSQL stock table
- GET /v1/cart, DELETE /v1/cart/item/{id}, DELETE /v1/cart (with hold release)
- Load test: 50 concurrent scans → verify <100ms P99 on staging

Week 6: Payment Service (99.99% uptime target)

- POST /v1/payment/initiate: checkout_lock SETNX → cart snapshot → price revalidate (server-side, ± 0.01 tolerance) → idempotency INSERT → gateway order
- POST /v1/payment/webhook/razorpay: verifyRazorpaySignature FIRST → ON CONFLICT DO NOTHING → async processWebhookEvent() → 200 in <200ms
- Circuit breaker: RecordError() → calculate error rate over 60s sliding window → open at 5% → fallback Cashfree
- GET /v1/payment/status/{id}: customer polls this during payment processing screen
- Integration test: same idempotency_key twice → one charge. Same webhook twice → one order.

Week 7: Order Service (Kafka consumer)

- Consumer: payment.confirmed → load cart from Redis → INSERT orders + order_items (append-only, immutable)
- Issue exit JWT (exp=+10min) → write exit_preauth:{user_id} to Redis Sessions cluster
- INSERT exit_tokens with jti, expires_at → clear cart from Redis → publish order.completed
- Async: generateInvoicePDF() → S3 upload → UPDATE orders.invoice_s3_key
- Publish: loyalty.points_earned, notification.payment_receipt to Kafka

Week 8: Exit Validation Service

- POST /v1/exit/validate: Ed25519 verify → store scope check → ExitQRUseScript (SET NX) → exit_preauth check → RFID mode detect
- storeHasRFIDPad(): devices table check, 5min Redis cache. Returns false for all Phase 0 pilot stores (QR_ONLY mode).
- POST /v1/exit/rfid-confirm: 3 retries × 200ms backoff. Publish device.alarm on failure.
- logValidation(): async INSERT exit_validation_log for compliance and dispute resolution
- Security tests: used QR → ALARM. expired QR → INVALID. wrong store → ALARM + log.

Week 9: Notification + Loyalty Services

- notification-service: consume notification.payment_receipt → sendWhatsAppReceipt() with exit QR embedded
- notification-service: consume notification.otp → sendFCMPush() for payment confirmation

- loyalty-service: consume loyalty.points_earned → calculatePoints(amount, tierMultiplier) → INSERT loyalty_ledger
- loyalty-service: evaluateTierUpgrade() → if tier changes: publish loyalty.tier_changed, UPDATE loyalty_cards
- GET /v1/loyalty/balance, GET /v1/loyalty/history, POST /v1/loyalty/redeem

Week 10: Inventory + Warehouse Services (Retailer)

- inventory-service: POST /v1/inventory/grn → receive items → UPDATE stock.available_qty → publish inventory.low_stock if below reorder_point
- inventory-service: AVAILABLE→RESERVED→SOLD→RETURNED state machine
- warehouse-service: POST /v1/warehouse/po → purchase order with vendor → publish warehouse.grn_discrepancy on mismatch
- warehouse-service: POST /v1/warehouse/transfer → store-to-store transfer → publish warehouse.transfer_dispatched

Week 11: Retailer Auth + Admin Auth + Admin Store

- retailer-auth: staff OTP → STAFF JWT with role claim (MANAGER, CASHIER, SECURITY, STOCK_ASSOCIATE)
- admin-auth: SSO login + TOTP 2FA → ZIPPYRA_ADMIN JWT. totp.go using pquerna/otp.
- admin-store: 10-step onboarding wizard API endpoints. Status machine: DRAFT→CONFIGURED→ACTIVE.
- admin-store: device registration → MQTT message to AWS IoT Core → device registers certificate
- compliance-service: consume account.deletion_requested → 30-day schedule → nullify PII fields across all tables

Week 12: Analytics + Chain HQ + Support

- analytics-service: consume order.completed → ClickHouse INSERT sales_summary
- analytics-service: consume cart.item_scanned → co_purchase_matrix update (batch)
- chain-hq-service: GET /chain/{id}/stores multi-store KPI, PUT /chain/{id}/config, POST /chain/erp/sync
- support-service: CRUD tickets + messages. Role-aware: customer sees own tickets, agent sees assigned.

9.3 Phase 0 — Apps & Polish (Weeks 13–18)

Week 13: Customer App — Auth + Store Entry

- Auth screens: phone entry (numpad) + OTP (6-digit auto-advance) + version check
- WatermelonDB setup: schema v11, models, migrations v1→v4
- POST /store/bind flow: expo-camera QR scan → bind API → store JWT → catalog sync
- useSync() with ±30s jitter. Conditional ETag. SQLite integrity_check on launch.
- Scan screen: react-native-vision-camera frame processor. Product popup with offer badge.

Week 14: Customer App — Cart + Payment + Exit

- Cart screen: live totals, offer banners, GST breakdown, proceed to checkout
- Checkout screen: CartSummary + PaymentSheet (Razorpay SDK native sheet)
- idempotency.ts: generateIdempotencyKey() persisted in expo-secure-store per cart
- payment/processing.tsx: poll every 2s, 30s timeout, then retry/support option

- payment/success.tsx: receipt + ExitQRCode.tsx full-screen + countdown timer
- exit/qrcode.tsx: 10min countdown, GateAnimation.tsx on success

Week 15: Retailer Dashboard — Core Screens

- Next.js PWA: inventory table, GRN workflow (scan barcodes on mobile too), staff management
- Real-time alerts via SSE (Server-Sent Events): low stock + device offline
- ClickHouse charts: SalesChart, PeakHoursChart, ShrinkageChart
- Order history with refund initiation → refund.requested Kafka event
- Device heartbeat status board: color-coded online/offline

Week 16: Admin Platform + Staff App

- admin.zippyra.com: store onboarding wizard (10 steps) with progress state saved server-side
- hq.zippyra.com: multi-store KPI dashboard using ClickHouse + Recharts
- Staff App: (tabs) home/inventory/grn/orders/alerts + RFID deactivation screen
- Electron Kiosk: idle → phone entry → OTP → scan → pay → receipt → reset

Week 17: EAS Build + Internal Testing

- EAS Build: Android .apk (internal track) + iOS .ipa (TestFlight)
- Internal test: 5 team members go through full scan→pay→exit in pilot store 1
- Fix all bugs found. Regression test: run all integration tests.
- Performance baseline: cart scan P99, payment confirm time, exit validation time

Week 18: Pilot Go-Live

- All 5 pilot stores: run zippyra-admin verify-deps → ALL 7 deps PASS
- Load test: k6 script — 50 concurrent scan→pay→exit per store
- RDS Multi-AZ failover test: kill primary → verify reconnect <30s
- Redis failover test: kill primary → verify session continuity
- Razorpay webhook end-to-end: real test payment from Razorpay dashboard
- All 3 Grafana dashboards showing live data from smoke test transactions
- Test PagerDuty alert: manually trigger high error rate → verify on-call receives P1
- On-call runbook complete. Incident response plan documented and reviewed.
- Soft launch: 5 pilot stores, invite-only, monitor for 1 week before public



Pre-Pilot Go-Live Checklist

10.1 Security Checks

Test	Expected	How to Test
Webhook invalid signature	HTTP 400 immediately	Send POST /webhook/razorpay with wrong X-Razorpay-Signature header
JWT blacklist eviction	HTTP 401 on revoked token	Logout, then use old token → must 401
Rate limit 6th OTP	HTTP 429 TooManyRequests	Send 6 POST /otp/send in <60s
STAFF JWT on customer endpoint	HTTP 403 Forbidden	Use staff JWT on POST /cart/scan → must 403
CUSTOMER JWT on staff endpoint	HTTP 403 Forbidden	Use customer JWT on POST /inventory/grn → must 403
Exit QR reuse	Result: ALARM, gate stays closed	Scan same exit QR twice within 10 min
Expired exit QR	Result: QR_EXPIRED (not ALARM)	Wait 11 min then try exit QR
Wrong store exit QR	Result: ALARM + logged	Scan store A's exit QR at store B gate
Cart price manipulation	HTTP 409 PRICE_CHANGED	POST /payment/initiate with total > server-calculated
Concurrent checkout	Only 1 succeeds, 2nd gets 409	Two simultaneous POST /payment/initiate from same user

10.2 Payment Integrity

Scenario	Expected	Verification
Same idempotency_key twice	1 payment row created	SELECT COUNT(*) FROM payments WHERE idempotency_key=X → must be 1
Same webhook event twice	1 order created	SELECT COUNT(*) FROM orders WHERE cart_session_id=X → must be 1
App killed mid-payment	Cart intact, resumable	Kill app during processing screen. Reopen → cart still there. Same idempotency_key resumes.
Razorpay down	Circuit opens → Cashfree fallback	Set RAZORPAY_KEY_ID to invalid. Verify circuit_breaker opens and Cashfree used.
Order Service DLQ	Retry 3× then DLQ alert	Stop order-service, trigger payment.confirmed event. Verify DLQ alert fires.
Invoice PDF failure	Order still CONFIRMED	Simulate S3 error. Verify order.status=CONFIRMED even if invoice_s3_key null.

10.3 Data Readiness

Check	Command	Pass Criteria
-------	---------	---------------

HSN/GST rates	SELECT COUNT(*) FROM hsn_gst_rates;	≥ 80 (pilot). ≥ 12,000 (production)
App versions seeded	SELECT * FROM app_versions;	2 rows: android + ios
Store QR tokens active	SELECT COUNT(*) FROM store_qr_tokens WHERE store_id=\$1 AND expires_at > NOW() AND is_used=false;	≥ 1 active token
Products imported	SELECT COUNT(*) FROM products WHERE store_id=\$1 AND is_active=true;	> 0 products
Offer rules in Redis	redis-cli -c GET offer_rules:{store_id}	Returns valid JSON (not nil)
SKU cache warm	redis-cli -c GET sku:{top_barcode}:{store_id}	Returns product JSON
All 7 deps	zippyra-admin verify-deps --store-id	ALL PASS — exit code 0
Razorpay webhook	Send test event from Razorpay dashboard	payment_webhook_events INSERT confirmed
AWS IoT Core	Device heartbeat MQTT message published	device heartbeat key in Redis Analytics cluster

10.4 Infrastructure Checks

Test	How	Pass Criteria
RDS Multi-AZ failover	AWS Console → RDS → Reboot with failover	App reconnects in <30s (pgxpool 3-retry with backoff)
Redis failover	AWS Console → ElastiCache → Failover primary	Session data survives. Cart holds survive (TTL 2h).
Kafka broker restart	kubectrl delete pod kafka-0 in MSK	Consumer group rebalances. No message loss (acks=all).
Pod OOMKilled	kubectrl top pod → identify large pod → kubectrl delete	HPA replaces within 30s. maxUnavailable=0 means no downtime.
S3 bucket access	aws s3 ls s3://zippyra-invoices/	Permission denied from public. Access only via IAM role.
Secrets rotation test	Rotate DB password in Secrets Manager → rolling restart	All pods pick up new password from CSI driver within 2 min.
CDN product images	GET https://cdn.zippyra.com/products/{store_id}/{sku_id}/test.jpg	Returns 200 with cache headers (CloudFront)

10.5 Observability Checks

Check	Pass Criteria
Pilot Store Health dashboard	All panels showing data. No 'No Data' panels.
Payment Gateway Health dashboard	gateway_health metrics visible. Circuit state shown.
Core Flow Funnel dashboard	scan→pay→exit funnel counts updating during smoke test.
PagerDuty P1 test	Manually set error rate >1% → PagerDuty P1 fires within 2 min.

Full OTEL trace	Search Jaeger for trace_id from scan → 5 services linked: cart→payment→order→exit→notification.
Kafka DLQ depth	kafka_consumer_group_lag{topic=~'*.dlq'} == 0 in Prometheus.
Log aggregation	CloudWatch Logs: /zippyra/payment-service → structured JSON logs visible.
Synthetic monitor	Grafana synthetic: auth endpoint check PASSING from Mumbai + Singapore.
Sentry integration	Trigger a test crash in customer app → verify Sentry alert with full stack trace.

■ ALL 5 CHECKLIST CATEGORIES CLEAR = READY FOR PILOT LAUNCH